

Analyse Technique : Logique Frontend (script.js)

Ce document dissèque le fichier script.js, le moteur interactif de **GeoMeteo**. Ce script ne se contente pas d'afficher des données ; il implémente des concepts avancés de gestion d'état, de résilience réseau et d'accessibilité (A11Y).

0. Helpers : Sécurité & Manipulation du DOM

```
function el(tag, props = {}, children = []) {  
  const node = document.createElement(tag);  
  Object.assign(node, props);  
  // ...  
}
```

Analyse Technique

Cette fonction simule une version légère du moteur de rendu de React. Elle permet de construire des éléments HTML de manière programmatique en gérant les propriétés et les enfants.

Pourquoi ce choix ?

- **Sécurité Anti-XSS** : En utilisant `document.createElement` au lieu de `innerHTML`, nous neutralisons tout risque d'injection de code malveillant. Les données sont traitées comme du texte pur.
- **Propreté du code** : Cela évite la manipulation de longues chaînes de caractères HTML illisibles et fragiles.

1. Accessibilité & Menu Mobile

```
mobileMenu.setAttribute('aria-expanded', String(isOpen));  
document.addEventListener('keydown', (e) => {  
  if (e.key === 'Escape') closeMenu();  
});
```

Analyse Technique

Le menu mobile gère non seulement les classes CSS pour l'animation, mais aussi les attributs ARIA et les événements clavier.

Pourquoi ce choix ?

- **Standard A11Y** : L'utilisation de aria-expanded permet aux malvoyants utilisant des lecteurs d'écran de savoir si le menu est ouvert.
- **Ergonomie** : L'écoute de la touche Escape est une convention UX attendue sur le web moderne pour fermer les éléments superposés.

2. Gestion de l'État & Configuration

```
const state = { requestSeq: 0, activeController: null };
const CONFIG = { TIMEOUT_MS: 12000, ... };
```

Analyse Technique

L'état de l'application est centralisé dans un objet state et les paramètres globaux dans un objet CONFIG.

Pourquoi ce choix ?

- **Maintenabilité** : Tous les "nombres magiques" (délais, quotas) sont au même endroit. Si l'on souhaite passer le cache de 2 à 5 minutes, cela ne prend qu'une seconde.
- **Prévisibilité** : Le compteur requestSeq permet de dater chaque requête pour ignorer les réponses obsolètes si l'utilisateur change de recherche très rapidement.

3. Communication Réseau "Senior" (Résilience)

```
async function fetchJsonWithTimeout(url, { signal } = {}) {
  const controller = new AbortController();
  // ... logic de timeout manuel + Exponential Backoff
}
```

Analyse Technique

Le script implémente un système de fetch augmenté qui gère les annulations, les timeouts et les tentatives de réessai.

Pourquoi ce choix ?

- **Économie de ressources** : L'usage de AbortController permet d'annuler physiquement les requêtes inutiles (si l'utilisateur tape vite), économisant la batterie du client et le CPU du serveur.
- **Résilience** : Le **Backoff Exponentiel** (attendre 0.5s, puis 1s, puis 2s) est une technique d'ingénierie réseau qui évite de saturer un serveur déjà en difficulté.

4. Rendu de l'Interface (Data-Driven UI)

```
function renderWeather(data) {
  const cards = [ { emoji: '🌡️', label: 'Température', ... }, ... ];
```

```
for (const c of cards) { ... }  
}
```

Analyse Technique

Le rendu des cartes météo n'est pas codé en dur mais généré par une boucle sur un tableau d'objets.

Pourquoi ce choix ?

- **Évolutivité** : Si nous décidons demain d'ajouter l'indice UV ou la qualité de l'air, il suffit d'ajouter une ligne dans le tableau cards sans toucher au moteur de rendu.
- **Cohérence** : Cela garantit que toutes les cartes ont exactement la même structure et le même comportement.

5. Persistance & Deep Linking

```
history.replaceState(null, "", `#city=${value}`);  
saveLastCity(city);
```

Analyse Technique

L'application utilise le localStorage pour la mémoire à long terme et l'URL hash pour l'état immédiat.

Pourquoi ce choix ?

- **Rétention** : L'utilisateur retrouve sa dernière recherche au redémarrage, créant une expérience fluide.
- **Partageabilité** : En mettant la ville dans l'URL (#city=Paris), l'utilisateur peut copier le lien et l'envoyer à un ami qui verra exactement le même résultat.

6. Autocomplétion Optimisée (Performance)

```
debounceTimer = setTimeout(async () => {  
  // ... fetch autocomplete ...  
}, 300);
```

Analyse Technique

L'autocomplétion utilise un **Debounce** de 300ms et mémorise les coordonnées GPS des suggestions.

Pourquoi ce choix ?

- **Performance Serveur** : On n'envoie pas une requête à chaque touche pressée, mais

seulement quand l'utilisateur a fini de taper.

- **Précision GPS** : En sauvegardant lat et lon lors de la suggestion, le script envoie des coordonnées précises au backend, ce qui est beaucoup plus fiable qu'une recherche par nom de ville pur.

Ce script démontre une maîtrise avancée de l'asynchronisme en JavaScript et une attention particulière portée aux détails qui font la différence en production.